

C# - Phần 10.11: Bài tổng hợp LINQ

Bài thực hành cuối Chương 10 - Đề bài, lời giải chuẩn, lý thuyết tổng kết và đánh giá

Mục tiêu: Tổng hợp toàn bộ LINQ đã học: Where, Select, FirstOrDefault, Any, All, Count, Sum, Average, Min, Max, OrderBy, ThenBy, Skip, Take, Distinct, GroupBy, Join và ToList.

PHẦN A - DỮ LIỆU BÀI TẬP

```
public class Category
{
    public string Id { get; }
    public string Name { get; }

    public Category(string id, string name)
    {
        Id = id;
        Name = name;
    }

    public override string ToString()
    {
        return $"{Id} - {Name}";
    }
}

public class Product
{
    public string Id { get; }
    public string Name { get; }
    public string CategoryId { get; }
    public decimal Price { get; }
    public int Stock { get; }

    public Product(
        string id,
        string name,
        string categoryId,
        decimal price,
        int stock)
    {
        Id = id;
        Name = name;
        CategoryId = categoryId;
        Price = price;
        Stock = stock;
    }

    public override string ToString()
    {
        return $"{Id} - {Name} - {Price} - Stock: {Stock}";
    }
}

List<Category> categories = new List<Category>
{
    new Category("C01", "Accessories"),
    new Category("C02", "Computer"),
    new Category("C03", "Storage")
};

List<Product> products = new List<Product>
{
    new Product("P01", "Mouse", "C01", 300, 10),
    new Product("P02", "Keyboard", "C01", 700, 5),
    new Product("P03", "Monitor", "C02", 2500, 3),
    new Product("P04", "Laptop", "C02", 15000, 2),
    new Product("P05", "USB", "C03", 150, 20),
    new Product("P06", "Cable", "C01", 300, 0),
    new Product("P07", "SSD", "C03", 1800, 4),
    new Product("P08", "Webcam", "C01", 1200, 6),
```

```

new Product("P09", "HDD", "C03", 1200, 7),
new Product("P10", "Laptop Stand", "C01", 700, 8)
};

```

PHẦN B - ĐỀ BÀI THỰC HÀNH

- 1. Lấy tất cả Product có Price $\geq$ 700 và Stock $>$ 0.
- 2. Từ tất cả Product còn hàng, chỉ lấy Name thành collection mới.
- 3. Tìm Product có Id = "P04". Nếu không tồn tại thì thông báo.
- 4. Kiểm tra có Product Stock == 0 hay không; kiểm tra tất cả Product có Price $>$ 0 hay không.
- 5. Thống kê tổng số Product, số Product còn hàng, tổng Stock, giá trung bình, giá thấp nhất, giá cao nhất.
- 6. Sắp Product theo Price giảm dần; nếu cùng Price thì Name tăng dần.
- 7. Phân trang sau khi sắp theo Id với pageNumber = 2, pageSize = 3.
- 8. Lấy các mức Price không trùng và sắp tăng dần.
- 9. Group theo CategoryId; mỗi group hiển thị CategoryId, Count, tổng Stock, giá trung bình.
- 10. Chỉ giữ những group có ít nhất 3 Product.
- 11. Join Product.CategoryId với Category.Id; kết quả gồm ProductName, CategoryName, Price, Stock.
- 12. Từ kết quả Join, chỉ lấy Product còn hàng và sắp Price giảm dần.
- 13. Từ kết quả Join, lấy CategoryName không trùng.
- 14. Kiểm tra products và categories gốc không bị thay đổi.
- 15. Tự viết pipeline Where -> OrderBy -> Skip -> Take -> Select -> ToList và giải thích kiểu dữ liệu từng bước.

PHẦN C - LỜI GIẢI CODE CHUẨN

```

public class Program
{
    public static void Main(string[] args)
    {
        List<Category> categories = new List<Category>
        {
            new Category("C01", "Accessories"),
            new Category("C02", "Computer"),
            new Category("C03", "Storage")
        };

        List<Product> products = new List<Product>
        {
            new Product("P01", "Mouse", "C01", 300, 10),
            new Product("P02", "Keyboard", "C01", 700, 5),
            new Product("P03", "Monitor", "C02", 2500, 3),
            new Product("P04", "Laptop", "C02", 15000, 2),
            new Product("P05", "USB", "C03", 150, 20),
            new Product("P06", "Cable", "C01", 300, 0),
            new Product("P07", "SSD", "C03", 1800, 4),
            new Product("P08", "Webcam", "C01", 1200, 6),
            new Product("P09", "HDD", "C03", 1200, 7),
            new Product("P10", "Laptop Stand", "C01", 700, 8)
        };

        // 1
        List<Product> filteredProducts =
            products
                .Where(product =>
                    product.Price >= 700 &&
                    product.Stock > 0)
                .ToList();

        // 2
        List<string> availableProductNames =
            products
                .Where(product => product.Stock > 0)
                .Select(product => product.Name)
                .ToList();

        // 3
        Product? foundProduct =
            products.FirstOrDefault(
                product => product.Id == "P04");
    }
}

```

```

if (foundProduct is not null)
{
    Console.WriteLine(foundProduct);
}
else
{
    Console.WriteLine("Product does not exist.");
}

// 4
bool hasOutOfStockProduct =
    products.Any(product => product.Stock == 0);

bool allPricesArePositive =
    products.All(product => product.Price > 0);

// 5
int totalProducts = products.Count();
int availableProducts =
    products.Count(product => product.Stock > 0);
int totalStock =
    products.Sum(product => product.Stock);
decimal averagePrice =
    products.Average(product => product.Price);
decimal minPrice =
    products.Min(product => product.Price);
decimal maxPrice =
    products.Max(product => product.Price);

// 6
var orderedProducts =
    products
        .OrderByDescending(product => product.Price)
        .ThenBy(product => product.Name);

// 7
int pageNumber = 2;
int pageSize = 3;

List<Product> pagedProducts =
    products
        .OrderBy(product => product.Id)
        .Skip((pageNumber - 1) * pageSize)
        .Take(pageSize)
        .ToList();

// 8
var distinctPrices =
    products
        .Select(product => product.Price)
        .Distinct()
        .OrderBy(price => price);

// 9
var productsByCategory =
    products.GroupBy(product => product.CategoryId);

foreach (var group in productsByCategory)
{
    Console.WriteLine($"Category: {group.Key}");
    Console.WriteLine($"Count: {group.Count()}");
    Console.WriteLine(
        $"Total stock: {group.Sum(product => product.Stock)}");
    Console.WriteLine(
        $"Average price: {group.Average(product => product.Price)}");
}

// 10
var largeGroups =
    productsByCategory
        .Where(group => group.Count() >= 3);

// 11
var productCategoryDetails =
    products.Join(

```

```

        categories,
        product => product.CategoryId,
        category => category.Id,
        (product, category) => new
        {
            ProductName = product.Name,
            CategoryName = category.Name,
            Price = product.Price,
            Stock = product.Stock
        });

// 12
var availableJoinedProducts =
    productCategoryDetails
        .Where(item => item.Stock > 0)
        .OrderByDescending(item => item.Price);

// 13
var distinctCategoryNames =
    productCategoryDetails
        .Select(item => item.CategoryName)
        .Distinct();

// 14
foreach (Product product in products)
{
    Console.WriteLine(product);
}

foreach (Category category in categories)
{
    Console.WriteLine(category);
}

// 15
List<string> productPipeline =
    products
        .Where(product => product.Stock > 0)
        .OrderBy(product => product.Price)
        .Skip(3)
        .Take(2)
        .Select(product => product.Name)
        .ToList();
}
}

```

PHẦN D - PHÂN TÍCH PIPELINE CÂU 15

```

products
→ List<Product>

.Where(product => product.Stock > 0)
→ IEnumerable<Product>

.OrderBy(product => product.Price)
→ IOrderedEnumerable<Product>

.Skip(3)
→ IEnumerable<Product>

.Take(2)
→ IEnumerable<Product>

.Select(product => product.Name)
→ IEnumerable<string>

.ToList()
→ List<string>

```

Ghi nhớ: Where lọc nhưng giữ kiểu phần tử; Select có thể đổi kiểu phần tử; OrderBy tạo IOrderedEnumerable; Skip/Take giữ kiểu phần tử; ToList vật hóa kết quả thành List.

PHẦN E - LÝ THUYẾT TỔNG KẾT CHƯƠNG 10

1. Where khác Select thế nào?

Where lọc phần tử theo điều kiện và thường giữ nguyên kiểu phần tử. Select biến đổi/chọn dữ liệu và có thể đổi kiểu phần tử.

2. Any khác Count thế nào?

Any trả bool để kiểm tra có ít nhất một phần tử thỏa điều kiện hay không. Count trả int để cho biết có bao nhiêu phần tử.

3. FirstOrDefault khác SingleOrDefault thế nào?

FirstOrDefault: 0 phần tử -> default; có nhiều -> lấy phần tử đầu tiên. SingleOrDefault: 0 -> default; 1 -> phần tử đó; >1 -> InvalidOperationException.

4. OrderBy + ThenBy hoạt động ra sao?

OrderBy là tiêu chí sắp xếp chính; ThenBy là tiêu chí phụ khi các phần tử bằng nhau theo khóa chính.

5. Pagination tính Skip thế nào?

Skip((pageNumber - 1) * pageSize). Sau đó Take(pageSize).

6. Distinct và GroupBy khác nhau thế nào?

Distinct loại trùng và giữ một đại diện; GroupBy gom các phần tử có cùng khóa vào từng group.

7. GroupBy tạo kiểu gì?

IEnumerable<IGrouping<TKey, TElement>>.

8. Join dùng các khóa như thế nào?

Join lấy khóa từ hai collection, so sánh các giá trị khóa có kiểu tương thích và ghép các phần tử khi khóa match.

9. Method nào thay đổi kiểu phần tử?

Select là method điển hình. GroupBy và Join cũng có thể tạo kiểu phần tử mới theo cấu trúc riêng.

10. Method nào chỉ lọc nhưng giữ kiểu phần tử?

Where.

PHẦN F - CÁC LỖI ĐÃ SỬA TRONG BÀI THỰC HÀNH

- Câu 2: phải lọc tất cả Product còn hàng từ products, không dùng lại tập đã lọc thêm Price >= 700.
- Câu 3: để yêu cầu tìm Product nên dùng FirstOrDefault(), không chỉ Any().
- Câu 5: sửa in nhầm biến Average và nhầm Max.
- Câu 10: bỏ Select(group => group.Key) nếu yêu cầu là giữ nguyên group.
- Câu 14: thực sự in/kiểm tra lại collection nguồn.
- Câu 15: Where phải là product => product.Stock > 0; đồng thời foreach phải duyệt productPipeline.
- Tên lambda trong Sum/Average nên dùng product thay vì trùng tên group.
- Xóa using không dùng đến.

PHẦN G - ĐÁNH GIÁ CUỐI CHƯƠNG

- Phần code sau khi sửa: khoảng 9.6/10.
- Phần lý thuyết tổng kết: khoảng 9.2/10.
- Đã chọn đúng LINQ method theo yêu cầu nghiệp vụ.
- Đã theo dõi được kiểu dữ liệu qua pipeline.
- Đã kết hợp được lọc, biến đổi, aggregate, sort, pagination, distinct, grouping và join trong cùng một bài.
- Cần tiếp tục chú ý đọc chính xác yêu cầu: kiểm tra tồn tại khác với lấy object; lọc group khác với chỉ lấy Key.

Kết luận: Chương 10 - LINQ: HOÀN THÀNH. Bài tiếp theo theo lộ trình là Chương 11 - Exception nâng cao, bắt đầu từ 11.1: hệ thống hóa try/catch/finally.